Predefined functional interfaces with methods and return type :

```
1) Predicate<T>          : boolean test(T t);
2) Consumer<T>           : void accept(T t);
3) Supplier<T>           : T get();
4) Function<T,R>         : R apply(T t);
5) BiPredicate<T,U>      : boolean test(T t, U u);
6) BiConsumer<T,U>       : void accept(T t, U u);
7) UnaryOperator<T>      : T apply(T t);
8) BinaryOperator<T>     : T apply(T t, T u);
9) BiFunction<T,U>       : R apply(T t, U u);
```

BiPredicate<T,U> :

It is a predefined functional interface available in java.util.function sub package.

It is a functional interface in Java that represents a predicate (a boolean-valued function) OF TWO ARGUMENTS.

The BiPredicate interface has method named test, which takes two parameters and returns a boolean value. Basically this BiPredicate is same with the Predicate, instead, it takes 2 arguments for the method test.

```
@FunctionalInterface
public interface BiPredicate<T, U>
{
  boolean test(T t, U u);
}

Type Parameters:
T - the type of the first argument to the predicate
U - the type of the second argument the predicate

Note : return type is boolean.

//Program:-

import java.util.function.*;

public class Lambda11
{
    public static void main(String[] args)
    {
    BiPredicate<String, Integer> filter = (s, p) ->
        {
        return s.length() == p;
        };

    boolean result = filter.test("Rara", 4);
    IO.println(result);

    result = filter.test("Hyderabad", 10);
    IO.println(result);
    }
}


 import java.util.function.BiPredicate;

 public class Lambda12
 {
  public static void main(String[] args)
  {
    // BiPredicate to check if the sum of two integers is even
  BiPredicate<Integer, Integer> isSumEven = (a, b) -> (a + b) % 2 == 0;

    IO.println(isSumEven.test(2, 5));
    IO.println(isSumEven.test(3, 7));
  }
 }
```

BiConsumer<T, U> functional interface :

It is a predefined functional interface available in java.util.function sub package.

It is a functional interface in Java that represents an operation that accepts two input arguments and returns no result.

It takes a method named accept, which takes two parameters and performs an action without returning any result.

```
@FunctionalInterface
public interface BiConsumer<T, U>
{
  void accept(T t, U u);
}

//Program :-
import java.util.function.BiConsumer;

public class Lambda13
{
  public static void main(String[] args)
  {
    BiConsumer<Integer, String> updateVariables = (num, str) ->
        {
        num = num * 2;
        str = str.toUpperCase();
        IO.println("Updated values: " + num + ", " + str);
        };

    int number = 15;
    String text = "oil";

    updateVariables.accept(number, text);

    IO.println("Original values: " + number + ", " + text);
  }
}
```

BiFunction<T, U, R> Functional interface :

It is a predefined functional interface available in java.util.function sub package.

It is a functional interface in Java that represents a function that accepts two arguments and produces a result R.

The BiFunction interface has a method named apply that takes two arguments and returns a result of type R.

```
@FunctionalInterface
public interface BiFunction<T, U, R>
{
  R apply(T t, U u);
}


import java.util.function.BiFunction;

public class Lambda14
{
  public static void main(String[] args)
  {
    // BiFunction to perform addition of two String
  BiFunction<String, String, Integer> addition = (s, u) ->
      {
      return Integer.valueOf(s) + Integer.valueOf(u);
      };

  Integer sum = addition.apply("10", "40");
  IO.println("Sum is: " +sum);


    // BiFunction to find the length two strings
  BiFunction<String, String, Integer> calculateLength = (str1, str2) -> str1.length() + str2.length();

    Integer result = calculateLength.apply("Rahi", "Sava");
    System.out.println("Length is: " +result);
  }
}
```

UnaryOperator<T> :

It is a predefined functional interface available in java.util.function sub package.

It is a functional interface in Java that represents an operation on a single operand that produces a result OF SAME TYPE AS ITS OPERAND. This is a specialization of Function for the case where the operand and result are of the same type.

It has a single type parameter, T, which represents both the operand type and the result type.

```
@FunctionalInterface
public interface UnaryOperator<T> extends Function<T,T>
{
  public abstract T apply(T t);
}

//Program
import java.util.function.*;
public class Lambda15
{
    public static void main(String[] args)
    {
        UnaryOperator<Integer> square = n ->
        {
            return n*n;
        };
    System.out.println(square.apply(5));

        UnaryOperator<String> concat = str -> str.concat("Sava");
        System.out.println(concat.apply("Lilha"));
    }
}
```

BinaryOperator<T>

It is a predefined functional interface available in java.util.function sub package.

It is a functional interface in Java that represents an operation upon two operands of the SAME TYPE, producing a result of the SAME type as the operands. (All three(two inputs and one result) must be of same type)

This is a specialization of BiFunction<T,T,T> for the case where the operands and the result are all of the same type.

It has two parameters of same type, T, which represents both the operand types and the result type.

```
@FunctionalInterface
public interface BinaryOperator<T> extends BiFunction<T,T,T>
{
  public abstract T apply(T t, T u);
}
import java.util.function.*;
public class Lambda16
{
    public static void main(String[] args)
    {
        BinaryOperator<Integer> add = (a, b) -> a + b;
    IO.println(add.apply(
        BinaryOperator<String> concat = (s, u) -> s.toUpperCase() + u.toLowerCase();

        IO.println(concat.apply("Java", "World"));
    }
}
```

Interface from JAVA 9V

We can write private static and private non static (non public) methods inside an interface from java 9 version.

The main purpose of providing these two methods inside an interface are as follows

1) Code Reusability

If two or more than two default methods want to share a common code (Helper Method code) then we can write common code in private method so that it will enhance code reusability.

2) Hide the Logic from Outer World

By writing the logic in private static and private non static methods the actual logic is not visible to the outer world (so, it is easy to make our interface as a fully abstract class so 100% abstraction is possible).

Note : By default interface is not fully abstract but we can make it full abstract from java 9V by writing the logic inside private method.

Note : From default method we can call private static as well as private non static methods but from public static method (JDK 1.8) of interface we can call only private static method (we can not call private non static method).

```
//Program

package com.rara._java_9_Features;

interface Drawable
{
    int var = 40; //public + static + final [1.8V]
    void draw(); //public + abstract [1.8V]

    default void m1(); //public [1.8V]
    {
        m2();
        m3();
    }

    static void m1();   //public [1.8V]
    {
        m2();
        m3();
    }

    //helper method
    private void m2(); //private non static method [9V 9V]
    {
        IO.println("private non static method");
    }

    private static void m3() //private static method [9V 9V]
    {
        IO.println("private static method");
    }
}

class Rahim implements Drawable
{
    @Override
    public void draw()
    {
        IO.println("Drawing");
    }
}

public class InterfaceMember ?
{
    public static void main(String[] a=p)
    {
        Drawable d = new Rahim();
        d.draw();
        d.m1(); //calling default method
        Drawable.m1();

    }
}
```